Junior Crypt CTF — Write-Up

Miscellaneous, OSINT, Cryptography, Forensics & Reverse Engineering

Contents

1 Challenge 1 — 1000-7 **2**

2 Challenge 2 — Fire Mural **3**

3 Challenge 3 — Invisible Editor **4**

4 Challenge 4 — Ghost Layers **5**

5 Challenge 5 — Aperture Science: Stream Calibration **6**

6 Challenge 6 — Aperture Science: Still Alive **7**

7 Challenge 7 — Aperture Science: Neurotoxin Diagnostics **9**

8 Challenge 8 — Aperture Science: Weighted Companion Cube **12**

9 Challenge 9 — Invoice Without A Bank **15**

10 Challenge 10 — The USB That Wouldn’t Repeat **15**

11 Challenge 11 — Philologist **16**

12 Challenge 12 — Write The “Кодэ” **18**

13 Challenge 13 — Prompt And Pretext **21**

14 Challenge 14 — WrongCube+ **23**

Challenge 1 — 1000-7

Category

Miscellaneous

Description

This is one of the miscellaneous challenge, that is hiding the data in the form of music - Musical Instrument Digital Interface.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Used the basic commands - exiftool, strings, cat, for grepping the flag and finding the hidden data in the form of metadata, but found nothing.

Step 2. Opened the challenge in Audacity, found some sort of pattern leading to MIDI pitch value analysis.

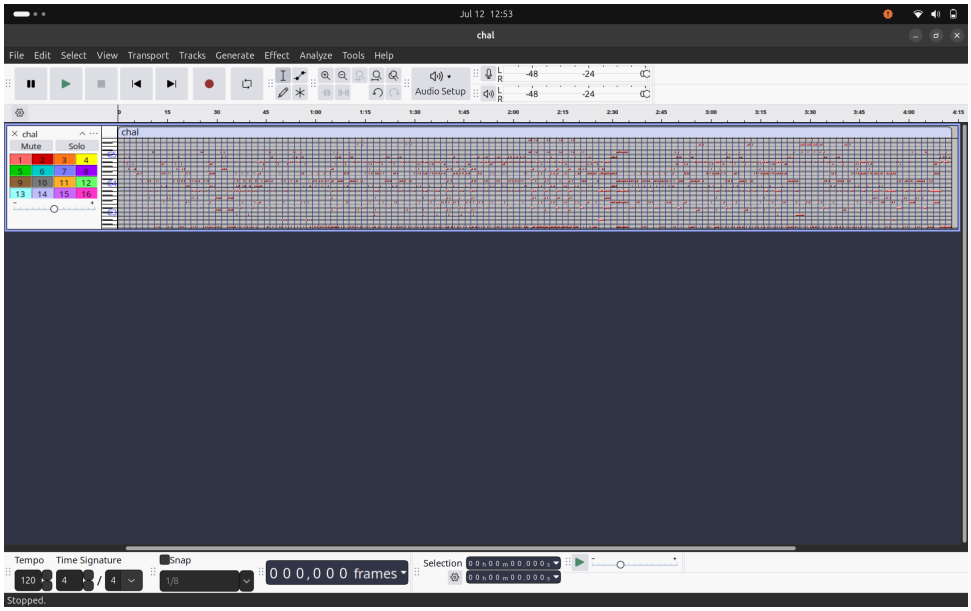


Figure 1: MIDI pitch value pattern observed in Audacity.

Step 3. After looking at the event dump, the pictchwheel events were osciallating with the high frequencies, the author used it for the binary encoding, high pitch – 1, low pitch – 0.

Step 4. Converted back into the ASCII characters, by grouping the binary into groups of 8.

Used the mentioned below code for the grouping and finding the correct sequence: -

```
import mido

mid = mido.MidiFile('chal.mid')
```

```
pitches = []

# 1. Extract all pitchwheel values in order
for track in mid.tracks:
    for msg in track:
        if msg.type == 'pitchwheel':
            pitches.append(msg.pitch)

# 2. Compress the Manchester pairs into single bits
pair_bits = []
for i in range(0, len(pitches), 2):
    pair = pitches[i:i+2]
    if len(pair) == 2:
        if pair[0] > 0:
            pair_bits.append(1)
        else:
            pair_bits.append(0)

print("--- Brute-forcing all possible text alignments ---")

# 3. Test shifts from 0 to 16 to find the exact flag boundary
for shift in range(16):
    shifted_bits = pair_bits[shift:]
    decoded_text = ""
    for i in range(0, len(shifted_bits), 8):
        byte = shifted_bits[i:i+8]
        if len(byte) == 8:
            val = int("".join(str(x) for x in byte), 2)
            # Use printable ASCII range, replace non-printables with dots
            if 32 <= val <= 126:
                decoded_text += chr(val)
            else:
                decoded_text += "."

    # Check if this alignment reveals the flag format
    if "grod" in decoded_text.lower():
        print(f"\n[SUCCESS] Found correct alignment at Shift {shift}:")
        print(decoded_text.replace(".", "")) # Strip out padding dots to
    show clean text
        print("-" * 40)
    else:
        print(f"Shift {shift}: {decoded_text[:30]}...")
```

Flag: `grodno{U1tr@_m3g@_5up3r_Gul_M1d_SF_1000-7}`

Challenge 2 — Fire Mural

Category

OSINT (Open Source Intelligence)

Description

This is one of the basic OSINT challenge, this is one of the creation that is present in Grodno and we have to identify the first and last name of the artist who created the monumental mural on the facade of this building.



Figure 2: Challenge image for Fire Mural.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Applied on the google image, and searched for the name of the artist, and got the flag.

Flag: `grodno{Vladimir Kachan}`

Challenge 3 — Invisible Editor

Category

Forensics

Description

This is one of the challenge, that is being asking for the hidden data that is present on the doc, shared as an attachment.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. `unzip invisible_editor.docx -d extracted_docx` – The doc file is having a zip file, the command is being used for the extraction of the zip file.

Step 2. `grep -ri "grodno" .` – This command is being used for the search

for the flag, in the entire doc.

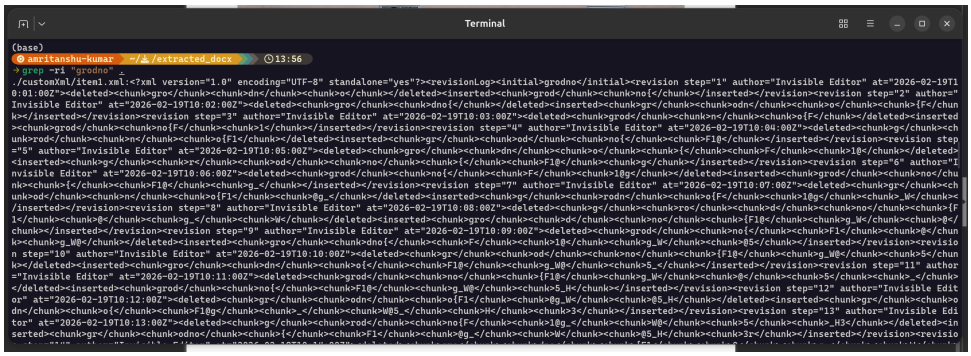


Figure 3: Grep search results inside the extracted docx contents.

Step 3. Looking closely to the text thar are being mentioned inside the tags, we are able to find the flag.

Flag: `grodno{F1@g_W@5_H3r3_0nc3}`

Challenge 4 — Ghost Layers

Category

Forensics / Steganography

Description

This is one of the challenge, that is being asking for the hidden message in the form of watermarks over the image shared as an attachment.

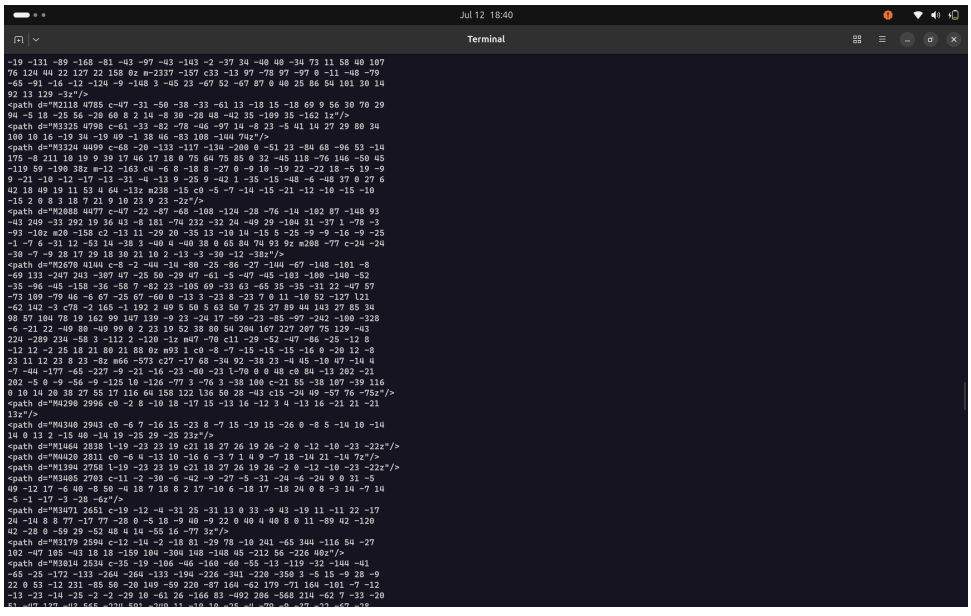


Figure 4: Challenge attachment for Ghost Layers.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Used some of the basic commands for finding the metadata, Used `strings beaver.svg`, found some pattern of text being mentioned in the form of various blocks, that gave a hint that something in the image is being mentioned in multilayer format.

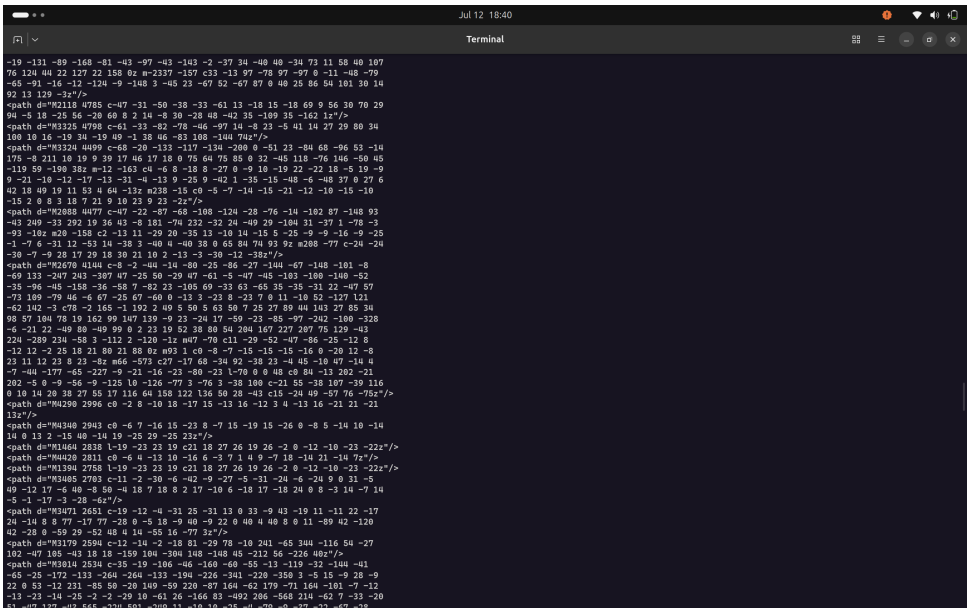


Figure 5: Pattern of text blocks found via `strings beaver.svg`.

Step 2. Used image – magic for the analysis of the image, found some of the text being visible after making the changes in the opacity of the image. Took the help of LLM for the analysis of the image and found the flag.

Flag: `grodno{9l0ry_t0_Al1v@r1@_9l0ry_t0_b33r}`

Challenge 5 — Aperture Science: Stream Calibration

Category

Cryptography

Description

This is one of the cryptography challenge, here we are being provided with the ciphertext, whose decryption is to be done.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. The ciphertext is been provided is consisting of hexadecimal values and are needed to be changed back with the help of the key to the plain text.

Step 2. Read the encoder.py, found the mentioned below things: -

- MOD, A, C — the exact LCG (Linear Congruential Generator) constants used to build the keystream
- The formula `state = (A * state + C) % MOD` — how each new keystream byte is generated
- That only `(state >> 24) & 0xFF` (the top byte of the 32-bit state) is used per output byte
- Critically: `state = seed & 0xFFFFF` — the seed gets masked down to just 20 bits before anything else happens

Step 3. Hence, in this case we did the brute force on a small target range, for the finding of the correct for the flag, used `plaintext = ciphertext XOR keystream`, this formula for the retrieval of the plain text.

Used the mentioned below code for brute forcing: -

```
MOD = 1 << 32
A = 1664525
C = 1013904223

def keystream(seed, length):
    state = seed & 0xFFFFF
    out = bytearray()
    for _ in range(length):
        state = (A * state + C) % MOD
        out.append((state >> 24) & 0xFF)
    return bytes(out)

ct = bytes.fromhex(open('ciphertext.txt').read().strip())

for seed in range(1 << 20):
    ks = keystream(seed, len(ct))
    pt = bytes(a ^ b for a, b in zip(ct, ks))
    if b'grodno{' in pt:
        print(seed, pt)
        break
```

Flag: `grodno{7h15_w45_4_7r1umph_bu7_7h3_533d_w45_700_5m4ll}`

Challenge 6 — Aperture Science: Still Alive

Category

Cryptography

Description

This is one of the cryptography challenge, a bit of continuation of the previous challenge. We are provided with a StillAlive.zip archive containing an RSA public key (public.pem) and a ciphertexts.json file. The goal is to recover the original plaintext message that was encrypted under this key,

without ever having access to the private key.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Inspected the RSA public key using the command: `openssl rsa -pubin -in public.pem -text -noout` This revealed a 2047-bit modulus N and, critically, a very small public exponent: e = 3. A low public exponent like this is a well-known RSA weakness when combined with certain message-encryption patterns.

Step 2. Examined ciphertexts.json and found the following structure:

- c1 = m1^3 mod N
- c2 = m2^3 mod N
- a = 1337
- b = <a large integer>

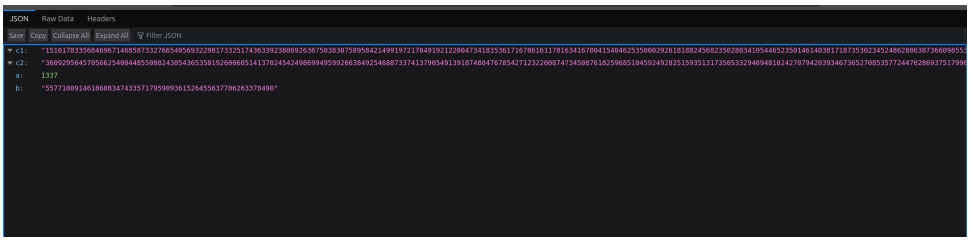


Figure 6: Contents of ciphertexts.json for Still Alive.

Step 3. The presence of a and b strongly suggested that the two original plaintext messages, m1 and m2, are linearly related to each other by the equation: `m2 = a*m1 + b (mod N)` – Found this suggestion with the help of LLM.

Step 4. Encrypting two related plaintexts under the same key with a small public exponent (e = 3) is exactly the vulnerable condition exploited by the Franklin–Reiter Related-Message Attack. This attack allows recovery of m1 directly from c1, c2, N, e, a, and b — without ever factoring N or knowing the private key.

Step 5. Constructed two polynomials over the ring Z_N[x]: f1(x) = x^3 − c1 (has m1 as a root, since m1^3 = c1) f2(x) = (ax + b)^3 − c2 (has m1 as a root, since substituting m2 = am1 + b makes this vanish)

Step 6. Both polynomials share the exact same root, m1. Computing the polynomial GCD of f1(x) and f2(x) modulo N collapses the two cubic polynomials down to a single linear factor of the form (x − m1).

Step 7. Implemented polynomial long division and the Euclidean GCD algorithm manually in Python, operating entirely with coefficients mod N. Ran the algorithm on f1(x) and f2(x) using the known values of c1, c2, a, b, and N.

Step 8. The GCD computation converged to a linear polynomial [g0, g1], from which m1 was recovered directly: m1 = (−g0 * inverse(g1)) mod N

Step 9. Converted the resulting integer m1 into bytes (m1.to_bytes(length, 'big')), which decoded cleanly into a readable plaintext message — a fake internal memo referencing GLaDOS — containing the flag in the expected grodno{} format, and got the flag.

Flag: `grodno{571ll_4l1v3_bu7_g14d05_k3375_r3w5171ng_m35s4g35}`

Challenge 7 — Aperture Science: Neurotoxin Diagnostics

Category

Cryptography

Description

We are provided with a NeurotoxinDiagnostics.zip archive stating that AES-CBC was used for encryption, along with only "packet timing telemetry" surviving the run and we are required to have the plaintext without having the AES key.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Read metadata.json, which stated: `mode = "aes-cbc"`, `block_size = 16`, note: "Only packet timing telemetry survived the diagnostic run."

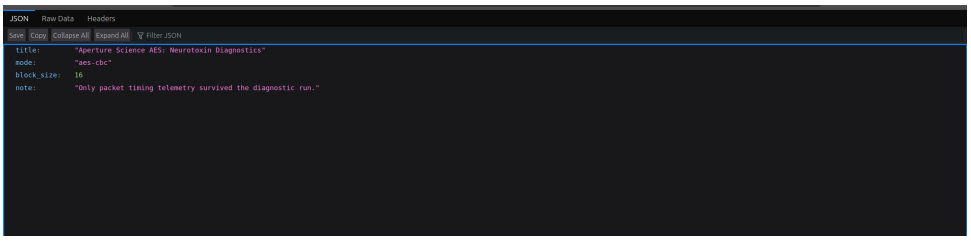


Figure 7: Contents of metadata.json for Neurotoxin Diagnostics.

This directed towards the CBC padding-oracle timing side-channel attack, and the challenge description also states that timing data is all that's available — no key, no plaintext.

Step 2. Parsed packet.hex using Python:

```
raw = bytes.fromhex(open('packet.hex').read().strip())
iv = raw[:16]
ct = raw[16:]
blocks = [ct[i*16:(i+1)*16] for i in range(8)]
```

Step 3. This gave a 16-byte IV followed by 8 ciphertext blocks of 16 bytes each (128 bytes total), consistent with standard AES-CBC struc-

ture.

Step 4. Checked timing_trace.json and found it contained 98,304 timing samples, each with the fields: block, pad, guess, trial, elapsed_ns. This confirmed the following things: -

- block: 1 to 8 (matching the 8 ciphertext blocks)
- pad: 1 to 16 (representing PKCS#7 padding length being targeted, byte by byte)
- guess: 0 to 255 (every possible byte value tried)
- trial: 0, 1, 2 (three repeated timing measurements per guess, to reduce noise)

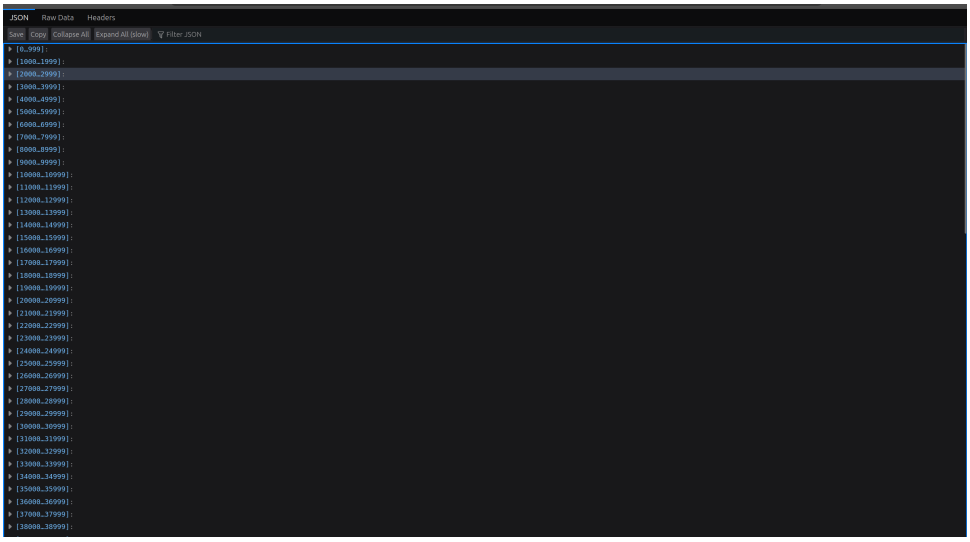


Figure 8: Structure of timing_trace.json samples.

Step 5. For each combination, averaged the elapsed_ns across the 3 trials for all 256 possible guess values, then identified the guess with the highest average time. In a padding oracle, a byte guess that produces valid padding forces additional processing on the server, so it takes measurably longer than the 255 invalid guesses. This was clearly visible in the data (e.g., for block 4, pad 4: guess 113 averaged ~2.74 million ns vs. a baseline of ~1.1–1.6 million ns for every other guess).

Step 6. Applied the standard CBC padding-oracle plaintext-recovery equations for each recovered guess:

```
intermediate_byte = correct_guess XOR pad
plaintext_byte = intermediate_byte XOR previous_ciphertext_block[
    position]
```

where position = 16 − pad (0-indexed from the start of the block), and previous_ciphertext_block is the IV for block 1, or the preceding real ciphertext block for blocks 2–8.

Step 7. Repeated this for all 8 blocks $\times$ 16 padding positions and assembled the full plaintext byte array.

Step 8. The recovered plaintext ended in valid PKCS - 7 padding bytes (\x03\x03\x03), confirming the attack was executed correctly, and decoded into a readable diagnostic record containing the flag.

With the help of LLM generated the code, and used the mentioned below code for the finding of the flag: -

```
import json, statistics
from collections import defaultdict

data = json.load(open('timing_trace.json'))
agg = defaultdict(list)
for d in data:
    agg[(d['block'], d['pad'], d['guess'])].append(d['elapsed_ns'])

hexstr = open('packet.hex').read().strip()
raw = bytes.fromhex(hexstr)
iv = raw[:16]
ct = raw[16:]
blocks = [ct[i*16:(i+1)*16] for i in range(8)]
prevs = [iv] + blocks[:-1]  # previous block for each of the 8 ciphertext
    blocks

plaintext = bytearray(8*16)

for block in range(1, 9):
    prev = prevs[block-1]
    for pad in range(1, 17):
        best_guess = None
        best_mean = -1
        for g in range(256):
            vals = agg[(block, pad, g)]
            m = statistics.mean(vals)
            if m > best_mean:
                best_mean = m
                best_guess = g

        pos = 16 - pad  # 0-indexed position within block
        intermediate = best_guess ^ pad
        pt_byte = intermediate ^ prev[pos]
        plaintext[(block-1)*16 + pos] = pt_byte

print(bytes(plaintext))
```

Flag: `grodno{n3ur070x1n_d14gn0571c5_l34k_7hr0ugh_71m1ng_4l0n3}`

Challenge 8 — Aperture Science: Weighted Companion Cube

Category

Cryptography

Description

We are being provided with a WeightedCompanionCube.zip archive containing several ”known” encrypted archive records, a catalog of possible field values, and one ”secret” encrypted archive to recover. The metadata explicitly notes that every archive in the batch was encrypted ”under the same boot-state”.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Read metadata.json, which described the encryption mode as ”aperture-companion-stream-v2” and gave the exact fixed record format:

```
"[Aperture Archive]"
"item=18"
"status=12"
"sector=12"
"memo=64"
```

This also gave a hint that every archive in this batch was encrypted under the same boot-state. This was the classic many-time pad flaw — if the same keystream is reused across multiple messages, then XORing any two ciphertexts cancels the keystream out entirely, leaving just the XOR of the two plaintexts.

Step 2. Read catalog.json, which contained a small closed list of only 6 possible values for each of the four fields (item, status, sector, memo) — meaning the plaintext for each known record was one of a very limited number of combinations, not arbitrary free text.

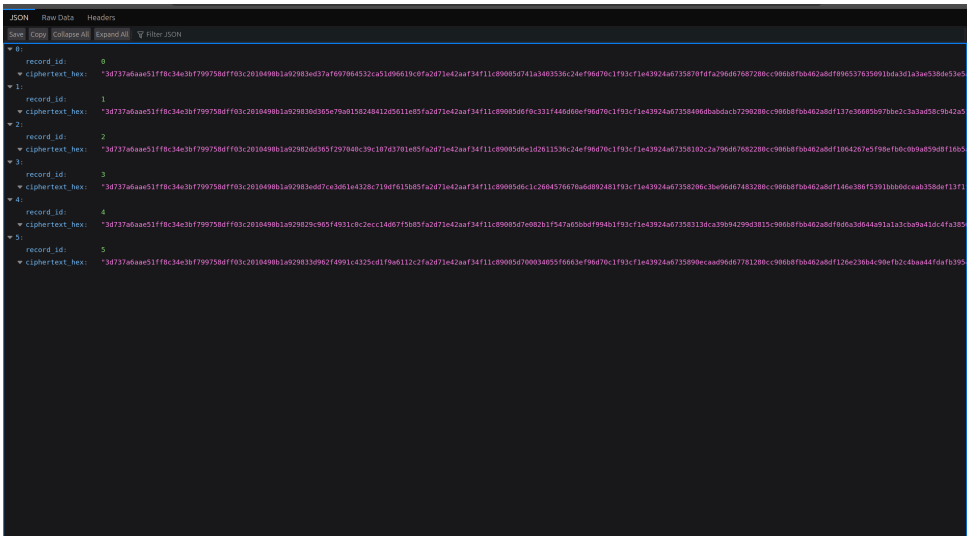


Figure 9: Contents of catalog.json for Weighted Companion Cube.

Step 3. Reconstructed the exact plaintext template from the format field widths given in metadata.json:

```
"[Aperture Archive]\nitem=" + (18 bytes) + "\nstatus=" + (12 bytes)
    + "\nsector=" + (12 bytes) + "\nmemo=" + (64 bytes) + "\n"
```

Verified this template totals exactly 153 bytes, matching every ciphertext's length.

Step 4. Recovered the keystream at every fixed position directly, since those plaintext bytes are identical and known in every record. Used this formula for the retrieval of the key stream.

```
keystream_byte = ciphertext_byte XOR known_literal_byte
```

Verified this derived keystream produced the exact same literal text when applied to all 6 known records — confirming they truly share one keystream.

Step 5. For the variable fields, brute – forced all 6 catalog values per field against all 6 records simultaneously, and for the memo field also did the same thing.

Used the mentioned below code for the finding of the flag, and brute – forcing: -

```
import json
from collections import defaultdict

ka = json.load(open('known_archives.json'))
cts = [bytes.fromhex(r['ciphertext_hex']) for r in ka]
catalog = json.load(open('catalog.json'))
secret = bytes.fromhex(open('secret_archive.hex').read().strip())

prefix = b"[Aperture Archive]\nitem="
```

```
item_w, status_w, sector_w, memo_w = 18, 12, 12, 64

p0 = len(prefix); p1 = p0 + item_w
mid1 = b"\nstatus="; p1b = p1 + len(mid1); p2 = p1b + status_w
mid2 = b"\nsector="; p2b = p2 + len(mid2); p3 = p2b + sector_w
mid3 = b"\nmemo="; p3b = p3 + len(mid3); p4 = p3b + memo_w
tail = b"\n"

item_pos, status_pos, sector_pos, memo_pos = (p0,p1), (p1b,p2), (p2b,p3), (
    p3b,p4)

ks_known = bytearray(len(cts[0]))
mask = bytearray(len(cts[0]))

def mark(lit, start):
    for i, b in enumerate(lit):
        ks_known[start+i] = cts[0][start+i] ^ b
        mask[start+i] = 1

# recover keystream at fixed literal positions
mark(prefix, 0)
mark(mid1, p1)
mark(mid2, p1b + status_w)
mark(mid3, p2b + sector_w)
mark(tail, p3b + memo_w)

def solve_field(pos, values, width):
    start, end = pos
    ks_votes = defaultdict(list)
    for ridx, c in enumerate(cts):
        for val in values:
            padded = val.encode() + b' ' * (width - len(val))
            ks = bytes(a ^ b for a, b in zip(c[start:end], padded))
            ks_votes[ks].append((ridx, val))
    for ks, lst in ks_votes.items():
        if len(set(r for r, v in lst)) == 6:
            return ks
    return None

for pos, values, width in [(item_pos, catalog['item'], item_w),
                           (status_pos, catalog['status'], status_w),
                           (sector_pos, catalog['sector'], sector_w),
                           (memo_pos, catalog['memo'], memo_w)]:
    start, end = pos
    ks = solve_field(pos, values, width)
    for i, b in enumerate(ks):
        ks_known[start+i] = b
        mask[start+i] = 1

pt = bytes(a ^ b for a, b in zip(secret, ks_known))
print(pt)
```

Flag: `grodno{c0mp4n10n_cub3_7h15_15_57r1c7ly_4_m4ny_71m3_p4d}`

Challenge 9 — Invoice Without A Bank

Category

Forensics

Description

This is one of the forensics challenge, in this we are being provided with various .eml file, an email archive, we have to look for the file_name and the subject_id that is being transferred.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. `grep -ri "Fatura Emitida" .` – Used this command for the finding of username that is being provided in the challenge description. This grep command gave the subject_id.

Step 2. `grep -i "filename" sample-717.eml` – Used this command for the retrieval of the filename.



Figure 10: Grep results revealing the filename inside the .eml file.

Flag: `grodno{Vl6s3kCIKaUvwaUAeY.pdf_6ZFYeMmltso}`

Challenge 10 — The USB That Wouldn’t Repeat

Category

Forensics

Description

This is one of the forensics challenge, in this we are required to search inside the MD5 verification code for the two disk images: -

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Went inside the files shared as an attachment, and used the grep command and some searching inside the file for the retrieval of the md5 hashes, and we got the flag.

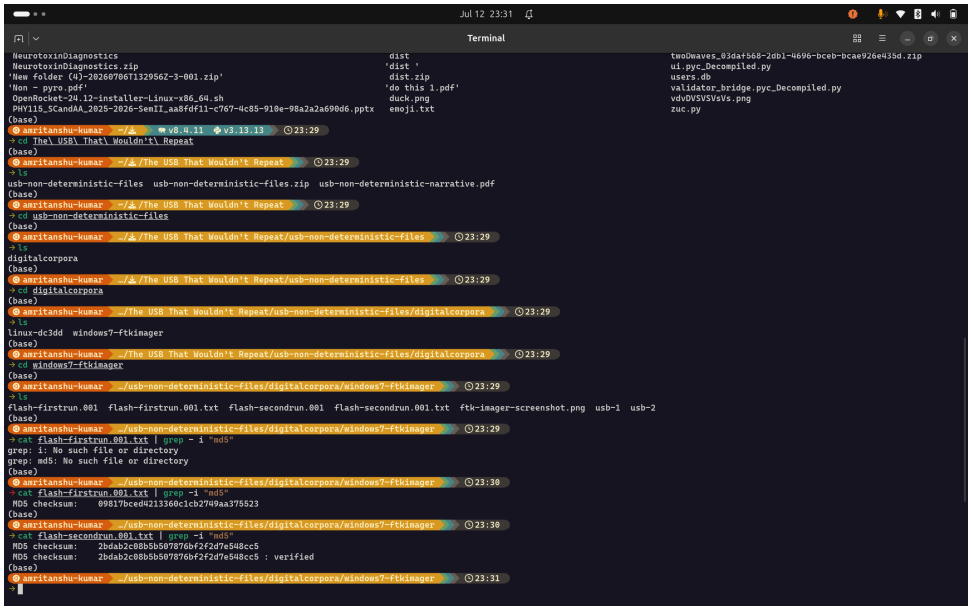


Figure 11: MD5 hashes retrieved from the disk image verification files.

Flag: `grodno{09817bced4213360c1cb2749aa375523_2bdab2c08b5b507876bf2f2d7e548cc5}`

Challenge 11 — Philologist

Category

Forensics

Description

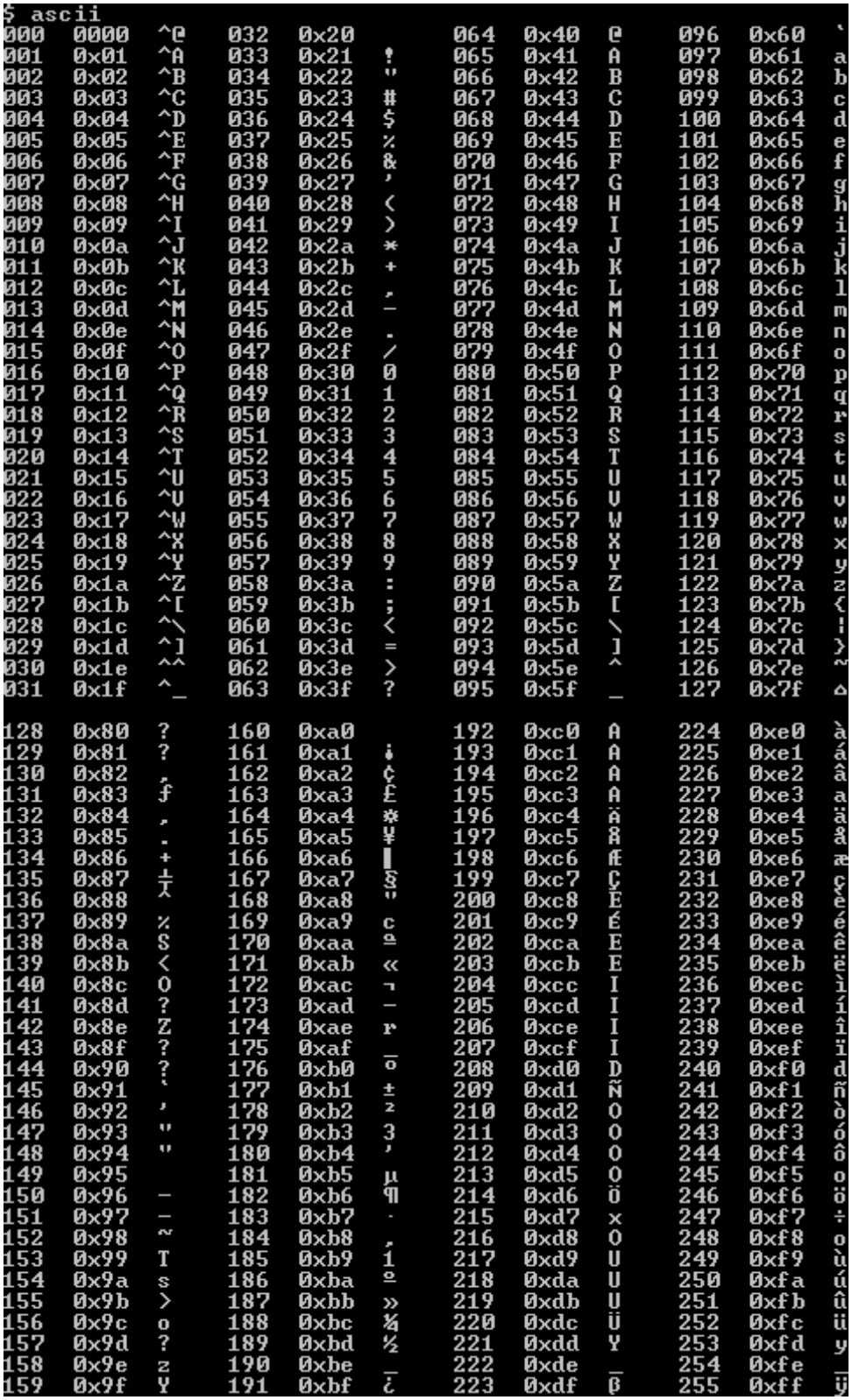
This is one of the forensics challenge, in which we are needed to look for the logs that are present inside the file and get the flag from there: -

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Inside the folder looked for the all gitlogs, the idea for the gitlog comes from the poem that is being mentioned on the challenge description.

Step 2. After checking the image and the formula file, that is being mentioned in the shared zip file, got the idea for the checking of the hexadecimal characters and converting them back into a possible flag.



```
$ ascii
000   0000  ^@    032  0x20        064  0x40  @     096  0x60  `
001   0x01  ^A    033  0x21  !     065  0x41  A     097  0x61  a
002   0x02  ^B    034  0x22  "     066  0x42  B     098  0x62  b
003   0x03  ^C    035  0x23  #     067  0x43  C     099  0x63  c
004   0x04  ^D    036  0x24  $     068  0x44  D     100  0x64  d
005   0x05  ^E    037  0x25  %     069  0x45  E     101  0x65  e
006   0x06  ^F    038  0x26  &     070  0x46  F     102  0x66  f
007   0x07  ^G    039  0x27  '     071  0x47  G     103  0x67  g
008   0x08  ^H    040  0x28  (     072  0x48  H     104  0x68  h
009   0x09  ^I    041  0x29  )     073  0x49  I     105  0x69  i
010   0x0a  ^J    042  0x2a  *     074  0x4a  J     106  0x6a  j
011   0x0b  ^K    043  0x2b  +     075  0x4b  K     107  0x6b  k
012   0x0c  ^L    044  0x2c  ,     076  0x4c  L     108  0x6c  l
013   0x0d  ^M    045  0x2d  -     077  0x4d  M     109  0x6d  m
014   0x0e  ^N    046  0x2e  .     078  0x4e  N     110  0x6e  n
015   0x0f  ^O    047  0x2f  /     079  0x4f  O     111  0x6f  o
016   0x10  ^P    048  0x30  0     080  0x50  P     112  0x70  p
017   0x11  ^Q    049  0x31  1     081  0x51  Q     113  0x71  q
018   0x12  ^R    050  0x32  2     082  0x52  R     114  0x72  r
019   0x13  ^S    051  0x33  3     083  0x53  S     115  0x73  s
020   0x14  ^T    052  0x34  4     084  0x54  T     116  0x74  t
021   0x15  ^U    053  0x35  5     085  0x55  U     117  0x75  u
022   0x16  ^V    054  0x36  6     086  0x56  V     118  0x76  v
023   0x17  ^W    055  0x37  7     087  0x57  W     119  0x77  w
024   0x18  ^X    056  0x38  8     088  0x58  X     120  0x78  x
025   0x19  ^Y    057  0x39  9     089  0x59  Y     121  0x79  y
026   0x1a  ^Z    058  0x3a  :     090  0x5a  Z     122  0x7a  z
027   0x1b  ^[    059  0x3b  ;     091  0x5b  [     123  0x7b  {
028   0x1c  ^\    060  0x3c  <     092  0x5c  \     124  0x7c  |
029   0x1d  ^]    061  0x3d  =     093  0x5d  ]     125  0x7d  }
030   0x1e  ^^    062  0x3e  >     094  0x5e  ^     126  0x7e  ~
031   0x1f  ^_    063  0x3f  ?     095  0x5f  _     127  0x7f  ⌂

128   0x80  ?     160  0xa0        192  0xc0  À     224  0xe0  à
129   0x81  ?     161  0xa1  ¡     193  0xc1  Á     225  0xe1  á
130   0x82  ‚     162  0xa2  ¢     194  0xc2  Â     226  0xe2  â
131   0x83  ƒ     163  0xa3  £     195  0xc3  Ã     227  0xe3  ã
132   0x84  „     164  0xa4  ¤     196  0xc4  Ä     228  0xe4  ä
133   0x85  …     165  0xa5  ¥     197  0xc5  Å     229  0xe5  å
134   0x86  †     166  0xa6  ¦     198  0xc6  Æ     230  0xe6  æ
135   0x87  ‡     167  0xa7  §     199  0xc7  Ç     231  0xe7  ç
136   0x88  ˆ     168  0xa8  ¨     200  0xc8  È     232  0xe8  è
137   0x89  ‰     169  0xa9  ©     201  0xc9  É     233  0xe9  é
138   0x8a  Š     170  0xaa  ª     202  0xca  Ê     234  0xea  ê
139   0x8b  ‹     171  0xab  «     203  0xcb  Ë     235  0xeb  ë
140   0x8c  Œ     172  0xac  ¬     204  0xcc  Ì     236  0xec  ì
141   0x8d  ?     173  0xad       205  0xcd  Í     237  0xed  í
142   0x8e  Ž     174  0xae  ®     206  0xce  Î     238  0xee  î
143   0x8f  ?     175  0xaf  ¯     207  0xcf  Ï     239  0xef  ï
144   0x90  ?     176  0xb0  °     208  0xd0  Ð     240  0xf0  ð
145   0x91  ‘     177  0xb1  ±     209  0xd1  Ñ     241  0xf1  ñ
146   0x92  ’     178  0xb2  ²     210  0xd2  Ò     242  0xf2  ò
147   0x93  "     179  0xb3  ³     211  0xd3  Ó     243  0xf3  ó
148   0x94  "     180  0xb4  ´     212  0xd4  Ô     244  0xf4  ô
149   0x95        181  0xb5  µ     213  0xd5  Õ     245  0xf5  õ
150   0x96  –     182  0xb6  ¶     214  0xd6  Ö     246  0xf6  ö
151   0x97  —     183  0xb7  ·     215  0xd7  ×     247  0xf7  ÷
152   0x98  ˜     184  0xb8  ¸     216  0xd8  Ø     248  0xf8  ø
153   0x99  ™     185  0xb9  ¹     217  0xd9  Ù     249  0xf9  ù
154   0x9a  š     186  0xba  º     218  0xda  Ú     250  0xfa  ú
155   0x9b  ›     187  0xbb  »     219  0xdb  Û     251  0xfb  û
156   0x9c  œ     188  0xbc  ¼     220  0xdc  Ü     252  0xfc  ü
157   0x9d  ?     189  0xbd  ½     221  0xdd  Ý     253  0xfd  ý
158   0x9e  ž     190  0xbe  ¾     222  0xde  Þ     254  0xfe  þ
159   0x9f  Ÿ     191  0xbf  ¿     223  0xdf  ß     255  0xff  ÿ
```

Figure 12: Image and formula file referenced in the shared zip.

Step 3. Used some of the basic git commands for the checking of the logs, tags, branches and deleted and changed logs.

Step 4. `git log -p` – Used this command for the checking of all logs of the

git even the deleted ones, looking closely to the name of the commits, re - assembled with the hexadecimal characters. The prefix of each of the commits gave a character for the flag, assembling all of them gave the flag.

Flag: `grodno{1o9f1a9}`

Challenge 12 — Write The “Кодэ”

Category

Reverse Engineering

Description

This is one of reverse challenge, in this we are required for to check, what is the logic that is being applied on the checker.c for the checking of the flag, and according to that generate the flag.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. `./tcc -B./runtime checker.c -o checker` - Compiled the checker.c file in the same manner as it is being stated in the description of the challenge.

Step 2. `strings -a -n 4 tcc | grep -A60 "int audit(const char"` – Used this for running strings on the tcc binary to pull out the leaked source of the injected function.

Step 3. This dumped the full C source of audit(), along with two helper functions and a static volatile unsigned char vm_blob[512] = {...} byte array — all left in as plain string data inside the compiler.

Step 4. After reading the logic inferred the following mentioned things: -

- relocation_key() opens /proc/self/exe, walks its own ELF SHT_RELA sections, and folds the relocation entries into a 64-bit key.
- next_word() is an xorshift64 PRNG driven by that key.
- vm_blob is XOR-decoded byte-by-byte with the PRNG stream into a small verification ”bytecode.”
- audit() then walks the input string one character at a time, mixing each byte into a running 32-bit checksum (XOR → rotate → add) and checking it against an expected value encoded in the bytecode.

Step 5. Extracted the 512-byte vm_blob array cleanly from the tcc strings dump into a file.

Step 6. With the help of LLM generated the code for the checking the ELF .rela sections and reproduce relocation_key(), for the regeneration of the xorshift64 keystream and decode vm_blob → bytecode and for tripling bytecode’s per-character and brute-force each char-

acter (0–255) against the known running state to recover the full expected string.

Step 7. `echo "grodno{Fabrice_Bellard_is_a_really_cool_programmer}" | ./checker` – This was used for checking the correctness of the flag.

Used the mentioned below code for the retrieval of the flag: -

```
import re
import struct

MASK32 = (1 << 32) - 1
MASK64 = (1 << 64) - 1

def rotl64(x, n):
    x &= MASK64
    return ((x << n) | (x >> (64 - n))) & MASK64

def relocation_key(path):
    """Reimplements tcc's embedded relocation_key():
    walks the target ELF's SHT_RELA sections and folds them into a 64-bit
   key."""
    with open(path, 'rb') as f:
        data = f.read()

    e_shoff, = struct.unpack_from('<Q', data, 0x28)
    e_shentsize, = struct.unpack_from('<H', data, 0x3A)
    e_shnum, = struct.unpack_from('<H', data, 0x3C)

    key = 0x6a09e667f3bcc909
    SHT_RELA = 4

    for i in range(e_shnum):
        off = e_shoff + i * e_shentsize
        sh_type, = struct.unpack_from('<I', data, off + 4)
        sh_offset, = struct.unpack_from('<Q', data, off + 24)
        sh_size, = struct.unpack_from('<Q', data, off + 32)
        if sh_type != SHT_RELA:
            continue
        relasz = 24  # sizeof(Elf64_Rela): r_offset(8) + r_info(8) +
   r_addend(8)
        count = sh_size // relasz
        for j in range(count):
            roff = sh_offset + j * relasz
            r_offset, r_info, r_addend = struct.unpack_from('<QQq', data,
   roff)
            key ^= (r_info + 0x9e3779b97f4a7c15 + (i << 32) + j) & MASK64
            key = (rotl64(key, 17) * 0xbf58476d1ce4e5b9) & MASK64
            key ^= r_addend & MASK64
    return key

def next_word(s):
    """xorshift64 stream generator (matches tcc's embedded next_word())."""
```

```
    s &= MASK64
    s ^= (s >> 12); s &= MASK64
    s ^= (s << 25); s &= MASK64
    s ^= (s >> 27); s &= MASK64
    val = (s * 0x2545f4914f6cdd1d) & MASK64
    return s, val

def extract_vm_blob(tcc_path):
    """Pull the literal vm_blob[512] initializer straight out of tcc's
    strings."""
    import subprocess
    out = subprocess.run(
        ["strings", "-a", "-n", "4", tcc_path],
        capture_output=True, text=True
    ).stdout
    line = next(l for l in out.splitlines() if "vm_blob[512]" in l)
    nums = re.findall(r'0x[0-9a-fA-F]{2}', line)
    assert len(nums) == 512, f"expected 512 bytes, got {len(nums)}"
    return bytes(int(x, 16) for x in nums)

def solve(checker_path="checker", tcc_path="tcc"):
    seed = relocation_key(checker_path)
    vm_blob = extract_vm_blob(tcc_path)

    # Decode vm_blob -> bytecode using the PRNG keystream
    code = bytearray(512)
    s = seed
    for i in range(512):
        s, val = next_word(s)
        code[i] = vm_blob[i] ^ ((val >> 56) & 0xff)

    n = code[0] | (code[1] << 8)
    assert code[2] == 0x51, "bad magic marker"

    pc = 3
    state = 0xC0DEC0DE
    answer = bytearray()

    for i in range(n):
        assert code[pc] == 0xA7, f"bad opcode at position {i}"
        pc += 1
        add_byte = code[pc]; pc += 1
        rot_byte = code[pc]; pc += 1
        want = (code[pc] | (code[pc + 1] << 8) |
                (code[pc + 2] << 16) | (code[pc + 3] << 24))
        pc += 4

        rot = rot_byte & 31
        inc = (0x9e3779b9 ^ ((i * 0x45d9f3b) & MASK32)) & MASK32

        found = None
```

```
        for ch in range(256):
            val = (ch + add_byte) & MASK32
            st = (state ^ val) & MASK32
            if rot:
                st = ((st << rot) | (st >> (32 - rot))) & MASK32
            st = (st + inc) & MASK32
            if st == want:
                found = ch
                new_state = st
                break

        if found is None:
            raise RuntimeError(f"no valid character found at position {i}")

        answer.append(found)
        state = new_state

    return bytes(answer)

if __name__ == "__main__":
    flag = solve()
    print(flag.decode())
```

Flag: `grodno{Fabrice_Bellard_is_a_really_cool_programmer}`

Challenge 13 — Prompt And Pretext

Category

Forensics

Description

This is one of the forensics challenge, here we are being provided with the Windows event log and are being asked for the checking of the same and recover the function that is asking for the credentials and the placeholder that left of the other actions.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Checked the file, and used some of the basic commands for the checking of the credentials present inside the zip.

Step 2. `grep -i "credential" evtx_data.csv | head -n 30`, `grep -iE "iex|encodedcommand|-enc|downloadstring" evtx_data.csv | head -n 30` – Used these commands for the checking of the logs of the windows, but found a massive log table with the data, being encoded in the form of base64 data.

Step 3. Inside the Credential Access, found the a different .evtx file `phish_windows_credentials_powershell_scriptblockLog_4104.evtx`, extracted

the text safely and found the required credentials.



Figure 13: Extracted PowerShell ScriptBlock log revealing the credentials.

Step 4. After peeling back the Base64 layer and decompressing the Gzip stream from that command, the complete underlying PowerShell script reveals the required credentials.

The mentioned below is the decrypted script: -

```
function Invoke-LoginPrompt{
$cred = $Host.ui.PromptForCredential("Windows Security", "Please enter user
    credentials", "$env:userdomain\$env:username","")
$username = "$env:username"
$domain = "$env:userdomain"
$full = "$domain" + "\" + "$username"
$password = $cred.GetNetworkCredential().password
Add-Type -assemblyname System.DirectoryServices.AccountManagement
$DS = New-Object System.DirectoryServices.AccountManagement.PrincipalContext
    ([System.DirectoryServices.AccountManagement.ContextType]::Machine)
while($DS.ValidateCredentials("$full","$password") -ne $True){
$cred = $Host.ui.PromptForCredential("Windows Security", "Invalid
    Credentials, Please try again", "$env:userdomain\$env:username","")
$username = "$env:username"
$domain = "$env:userdomain"
$full = "$domain" + "\" + "$username"
$password = $cred.GetNetworkCredential().password
Add-Type -assemblyname System.DirectoryServices.AccountManagement
$DS = New-Object System.DirectoryServices.AccountManagement.PrincipalContext
    ([System.DirectoryServices.AccountManagement.ContextType]::Machine)
$DS.ValidateCredentials("$full", "$password") | out-null
}
$output = $newcred = $cred.GetNetworkCredential() | select-object UserName,
    Domain, Password
$output
R{START_PROCESS}
}
Invoke-LoginPrompt
```

Flag: `grodno{Invoke-LoginPrompt_R{START_PROCESS}}`

Challenge 14 — WrongCube+

Category

Reverse Engineering

Description

This is a Reverse Engineering challenge, where we are provided with a compiled Windows executable (WrongCube+.exe) that simulates a Kubernetes-like cluster environment. The objective is to unpack the executable, decompile the underlying bridge scripts, and reverse-engineer the bundled C library to bypass the impossible cluster validation checks and recover the hidden flag.

Approach and Solution

Used the mentioned below steps and commands: -

Step 1. Extracted the Payload, used pyinstxtractor in a Python 3.12 environment to successfully unpack the executable and its PYZ.pyz dependencies without unmarshalling errors.

Step 2. `python pyinstxtractor.py WrongCube+.exe`

Step 3. Decompiled the Python Wrapper, used PyLingual to decompile main.pyc and src/validator_bridge.pyc. Discovered that the Python code that converts the cluster layout into JSON and passes it to a compiled C library (wrongkube_validator.dll).

Step 4. Loaded wrongkube_validator.dll into Ghidra and tracked down the exported validate_cluster function, did this for the reverse engineering of the .dll file.

Step 5. Analyzed the Validation Logic, traced the C pseudo-code to uncover the win condition: the program secretly required an exact 21-node and 40-edge cluster layout to trigger the flag generation.

Step 6. Bypassed the Application, found the located the flag decryption loop inside the C code.

Step 7. Decrypted the Flag, extracted the static encryption keys and wrote a Python script to scan the .dll file, replicate the XOR math, and dynamically decrypt the hidden flag from the binary's .data section.

Used the mentioned below code for the retrieval of the flag: -

```
import os

def generate_keystream():
    iVar11 = 0xF73449EF
    bVar52 = 0x49
    iVar50 = 0x47502943
    lVar19 = 1
    iVar12 = 0x47502932
    iVar48 = 0x3c6ef35f
```

```
    uVar30 = 0
    keystream = bytearray(46)

    while lVar19 != 0x2f:
        iVar13 = (iVar11 * 0x19660d + iVar48) & 0xFFFFFFFF
        k1 = ((bVar52 + 0xb7) & 0xFF) ^ ((iVar13 >> 16) & 0xFF) ^ (iVar13 &
    0xFF)
        keystream[lVar19 - 1] = k1

        iVar13_b = (iVar11 * 0x17385ca9) & 0xFFFFFFFF
        iVar11 = (uVar30 + 1 + ((uVar30 | 1) * 0x10) + iVar12 + iVar13_b) &
    0xFFFFFFFF

        iVar13_c = (iVar13_b + iVar50) & 0xFFFFFFFF
        k2 = bVar52 ^ ((iVar13_c >> 16) & 0xFF) ^ (iVar13_c & 0xFF)
        keystream[lVar19] = k2

        uVar30 = (uVar30 + 2) & 0xFFFFFFFF
        bVar52 = (bVar52 + 0x92) & 0xFF
        iVar50 = (iVar50 + 0x35f8ddc) & 0xFFFFFFFF
        lVar19 += 2
        iVar12 = (iVar12 + 0x35f8dba) & 0xFFFFFFFF
        iVar48 = (iVar48 + 0x22) & 0xFFFFFFFF

    return keystream

def solve():
    keystream = generate_keystream()

    dll_path = None
    for root, dirs, files in os.walk("."):
        if "wrongkube_validator.dll" in files:
            dll_path = os.path.join(root, "wrongkube_validator.dll")
            break

    if not dll_path:
        print("[!] Could not find wrongkube_validator.dll")
        return

    with open(dll_path, "rb") as f:
        data = f.read()

    print("[*] Scanning file for human-readable flags...")
    found = False
    for i in range(len(data) - 46):
        chunk = data[i:i+46]
        decrypted = bytearray(46)
        for j in range(46):
            decrypted[j] = chunk[j] ^ keystream[j]

        if b"{" in decrypted and b"}" in decrypted:
```

```
            # Count how many characters are standard printable ASCII
            printable_count = sum(1 for b in decrypted if 32 <= b <= 126)
            # The false positive had very few printables. The real flag will
     be almost entirely text.
            if printable_count > 30:
                # Replace any trailing padding bytes with a dot for clean
    terminal output
                text = "".join(chr(b) if 32 <= b <= 126 else "." for b in
    decrypted)
                print(f"\n[!!!] WINNER - FLAG FOUND AT FILE OFFSET {hex(i)}
    [!!!]")
                print(f"--> {text}\n")
                found = True

    if not found:
        print("[!] Could not find a readable flag. Mathematical logic might
    need a tweak.")

if __name__ == "__main__":
    solve()
```

Flag: grodno{5h4d0w_c0ntr0l_pl4n3_qu0rum_r3c0nc1l3d}

End of Write-up